

DTE Platform — Repository Bootstrap & Solution Skeleton

As the Principal .NET Architect and Repository Governance Lead, this document is the definitive blueprint for initializing the physical source code repository. It dictates the exact directory layout, project configurations, package whitelists, and CI/CD bootstrap required to execute the V1 Implementation Roadmap.

This document assumes the developer is executing these instructions to create the initial commit (Commit 0) of the platform.

1. Repository Philosophy

The physical structure of the repository is the first line of defense for the architecture.

- Folders map to architectural layers.
- .csproj references map to dependency rules.
- The presence of a package in a project file acts as a legally binding architectural contract.

We do not rely on developer discipline to prevent coupling; we rely on the .NET compiler and CI pipeline.

2. Repository Root Layout

The root of the repository must exactly match the following hierarchy.

Plaintext

```
dte-platform
├── github
│   └── workflows/      # CI/CD pipeline definitions
├── build/              # Build scripts and MSBuild properties (Directory.Build.props)
├── deploy/            # Dockerfiles, docker-compose, and infrastructure manifest
├── docs/              # Architectural Decision Records (ADRs) and onboarding guides
├── scripts/           # Local dev automation (e.g., bootstrap-db.ps1)
├── src/               # Production source code projects
├── tests/             # Automated test projects
├── tools/             # .NET local tool manifests (config/dotnet-tools.json)
├── dockerignore
├── editorconfig       # Mandatory formatting and analyzer rules
├── gitignore
├── DtePlatform.sln    # Master solution file
└── README.md
```

3. Solution Structure

The DtePlatform.sln will organize projects into virtual folders src and tests.

Project Reference Enforcement (Dependency Direction):

- DTE.Domain $\rightarrow$ DTE.Application
- DTE.Application $\rightarrow$ DTE.Infrastructure
- DTE.Application, DTE.Infrastructure $\rightarrow$ DTE.Api (Infrastructure referenced *only* for DI bootstrapping in Program.cs)

Required Assembly Boundaries:

Do not split Domain into Domain.Entities and Domain.Exceptions. Do not create a DTE.Common project. The boundary is the architectural layer.

4. Source Projects

All projects target net8.0.

DTE.Domain

- **Responsibility:** Ubiquitous language, Aggregates (FiscalDocument), Value Objects, Domain Events.
- **Allowed Dependencies:** None. Standard Library only.
- **Forbidden Dependencies:** Everything else.

DTE.Application

- **Responsibility:** Use Cases (MediatR Handlers), Interface definitions (IDocumentTransmissionQueue, IMhApiClient), FluentValidation rules.
- **Allowed Dependencies:** DTE.Domain, MediatR, FluentValidation.
- **Forbidden Dependencies:** Microsoft.EntityFrameworkCore, external SDKs.

DTE.Infrastructure

- **Responsibility:** EF Core DbContext, Hangfire job wrappers, PostgreSQL JSONB configurations, HTTP client implementations for MH API, Cryptographic signing wrappers.
- **Allowed Dependencies:** DTE.Domain, DTE.Application, Microsoft.EntityFrameworkCore, Npgsql, Hangfire, Serilog.
- **Forbidden Dependencies:** DTE.Api.

DTE.Api

- **Responsibility:** Kestrel host, ASP.NET Core Controllers, JWT Middleware, Exception-to-HTTP mapping.
- **Allowed Dependencies:** DTE.Application, DTE.Infrastructure (strictly for IServiceCollection extension methods).
- **Forbidden Dependencies:** Direct references to EF Core DbContext.

5. Test Projects

Test projects live in /tests and target net8.0.

- **DTE.Domain.Tests:** Unit tests using xUnit and FluentAssertions. Mocks are forbidden. Instantiates objects directly.
- **DTE.Application.Tests:** Tests CQRS handler logic. Mocks infrastructure interfaces (IMhApiClient).
- **DTE.Infrastructure.Tests:** Integration tests. Mocks are forbidden. **Requires Testcontainers** to spin up a physical PostgreSQL database. Validates EF Core mapping and DB-level triggers (e.g., Evidence immutability).
- **DTE.Api.Tests:** End-to-End tests utilizing WebApplicationFactory and Testcontainers. Calls HTTP endpoints and verifies JSON responses.
- **DTE.Architecture.Tests:** Validates the codebase using NetArchTest.Rules.

Coverage Expectations: Domain invariants and State Machine transitions must have 100% branch coverage.

6. NuGet Package Strategy

This is the explicit whitelist. Do not install packages outside this list without Architecture Board approval.

Approved Catalog:

- MediatR (CQRS pipeline in Application)
- FluentValidation (Input validation in Application)
- Microsoft.EntityFrameworkCore & Npgsql.EntityFrameworkCore.PostgreSQL (Persistence in Infrastructure)
- Hangfire.PostgreSql & Hangfire.AspNetCore (Background processing)
- Serilog.AspNetCore (Structured logging)
- NetArchTest.Rules (Architecture validation in Tests)
- Testcontainers.PostgreSql (Integration testing)
- xUnit, FluentAssertions, Moq or NSubstitute (Testing framework)

Forbidden Packages:

- Microsoft.EntityFrameworkCore.InMemory (Hides PostgreSQL JSONB constraints and triggers).
- AutoMapper (Obscures mapping logic; we use explicit mapping methods for transparency).
- Any generic repository NuGets.

7. Configuration Strategy

Configuration is strictly typed using the IOptions<T> pattern.

appsettings.json (Base Layout):

JSON

```

"Database" "ConnectionString" ""
"Hangfire" "WorkerCount" 4
"Security"
"JwtSecret" ""
"Issuer" "DtePlatformV1"
"Audience" "DtePlatformV1"

"MhApi" "BaseUrl" "https://api.mh.gob.sv" "Environment" "00"
"CertificateVault" "Provider" "LocalPath" "Reference" ""

```

Overrides:

- appsettings.Development.json: Used strictly for local DB endpoints.
- **Environment Variables:** All deployments override secrets using env vars prefixed with DTE__ (e.g., DTE__Database__ConnectionString).
- **Secrets:** Local development must use dotnet user-secrets. Committing passwords or .pfx pins to Git is grounds for access revocation.

8. Local Development Bootstrap

Required Tools:

- .NET 8 SDK
- Docker Desktop or OrbStack
- IDE: Visual Studio 2022, JetBrains Rider, or VS Code with C# Dev Kit.

Developer Workstation Setup:

1. Clone repo.
2. Run `docker-compose -f deploy/docker-compose.dev.yml up -d` (Spins up local PostgreSQL 15 on port 5432).
3. Run `dotnet restore DtePlatform.sln`.
4. Run `dotnet tool restore` (Restores dotnet-ef).
5. Run `dotnet run --project src/DTE.Api` (Runs Kestrel on localhost:5001).

9. Docker Strategy

deploy/Dockerfile:

Must use multi-stage builds.

- build stage: Uses `mcr.microsoft.com/dotnet/sdk:8.0`. Runs `dotnet publish`.
- runtime stage: Uses `mcr.microsoft.com/dotnet/aspnet:8.0-alpine` or `debian`.
- Security: App must run under a non-root user (app).

deploy/docker-compose.yml (Production Stack Prototype):

Defines api and db services.

- api: Depends on db via `depends_on`: `condition: service_healthy`.
- db: PostgreSQL 15. Mounts volume for `/var/lib/postgresql/data`.

10. CI/CD Bootstrap

Defined in .github/workflows/ci.yml.

Workflow Steps:

1. **Checkout Code.**
2. **Setup .NET 8 SDK.**
3. **Restore Dependencies.**
4. **Compile:** dotnet build --no-restore -c Release.
5. **Run Architecture Tests:** Executes DTE.Architecture.Tests first. Fails fast if boundaries are violated.
6. **Run Unit Tests:** Executes Domain and Application tests.
7. **Run Integration Tests:** Spins up Testcontainers. Executes Infrastructure and API tests.
8. **Verify EF Migrations:** Runs dotnet ef migrations bundle to ensure DbContext and Migrations compile successfully into a deployment artifact.

11. Logging Bootstrap

Configured in Program.cs immediately upon boot.

- **Provider:** Serilog.
- **Enrichment:** Enrich.FromLogContext().
- **Correlation ID Propagation:** Custom middleware intercepts HTTP requests, generates a UUID (or reads X-Correlation-ID), and pushes it to the LogContext.
- **Audit Correlation:** When AuditEntry is written to the DB, the same CorrelationId is attached to the Serilog operational logs, linking the system trace to the regulatory audit trail.

12. Database Bootstrap

- **Migration Strategy:** EF Core Code-First. Migrations live in DTE.Infrastructure.Migrations.
- **Hangfire Schema:** Hangfire initializes its own schema automatically during startup (UsePostgreSqlStorage).
- **Seed Strategy:** IHostedService runs on application startup. It checks for the existence of required MH Catalog entries (Cat-011, Cat-002, etc.). If missing, it parses embedded JSON files in DTE.Infrastructure and seeds the catalog_entry table.
- **Database Triggers:** EF Migrations MUST include raw migrationBuilder.Sql() calls to create the BEFORE UPDATE OR DELETE triggers on fiscal_document (post-signing), audit_entry, and evidence_artifact.

13. Security Bootstrap

- **Secrets Management:** Implemented via standard ASP.NET Core Configuration. In production, env vars map to K8s Secrets or Docker Swarm Secrets.
- **Signing Provider Abstraction:** Define ICertificateVault in DTE.Application.
- **Local Dev Implementation:** LocalFileCertificateVault reads a dummy .pfx from the local tests/certs folder.
- **Production Implementation:** Resolves the IssuerConfiguration.certificate_reference (e.g., Azure KeyVault URI, or absolute file path mounted securely to the container).

14. First Vertical Slice Implementation Sequence

The team must execute the "Walking Skeleton" (FE Draft $\rightarrow$ Accepted) in this exact order:

1. **DTE.Domain:** Create FiscalDocument aggregate. Implement Validate() method (ensuring Totals equal Lines).
2. **DTE.Application:** Create CreateDraftCommand, ValidateDocumentCommand, and MediatR handlers. Write Unit Tests.
3. **DTE.Infrastructure:** Map FiscalDocument to PostgreSQL (mapping snapshots to JSONB). Generate Migration 001.
4. **DTE.Api:** Create DocumentsController.cs. Wire endpoints to MediatR.
5. **DTE.Infrastructure:** Implement LocalFileCertificateVault.
6. **DTE.Application:** Implement SignDocumentCommand. Generates canonical JSON, calls vault, produces EvidenceBundle.
7. **DTE.Infrastructure:** Map EvidenceBundle (BYTEA). Write Migration 002 (with Triggers).
8. **DTE.Infrastructure:** Implement HangfireTransmissionQueue. Write MhApiClient.
9. **DTE.Api.Tests:** Write End-to-End Testcontainer test verifying the full lifecycle.

15. Repository Readiness Checklist

Do not accept the first production feature PR until this checklist is signed off:

- ☐ DtePlatform.sln compiles locally with 0 warnings.
- ☐ NetArchTest suite is written and passing.
- ☐ docker-compose.dev.yml successfully boots PostgreSQL.
- ☐ Testcontainers successfully spins up PostgreSQL and passes an empty DB test.
- ☐ CI pipeline (GitHub Actions) runs and passes on PR to main.
- ☐ .editorconfig is enforcing namespace conventions and code styling.
- ☐ EF Core Migration 000 (InitialCreate) is generated and applies cleanly to empty DB.